

ROADGUARD AI

Research & Development Report

Version 0.3 - Baseline, FP/FN Analysis, Field Testing, and Improvement Plan

Current project status

YOLO11s baseline integrated into the Streamlit prototype. Full validation completed on 3,833 images. FP/FN error analysis completed. Manual Saudi-road field testing identified UI and robustness issues that now guide the next controlled improvement experiment.

Prepared for the RoadGuard AI Capstone Team

6 September 2026

1. Executive Summary

This document organizes the RoadGuard AI R&D work into three development snapshots. The first established a correct baseline model integration, the second quantified model errors on the complete validation split, and the third adds manual field-testing evidence from the current Streamlit prototype.

Primary technical conclusion

The model is not mainly failing because it confuses classes. Its largest weakness is missed detections, especially D40 potholes. Field testing also suggests reduced reliability in visually complex/low-light scenes. A separate dark-mode UI visibility problem must be fixed independently from model training.

Version progression

Version	Focus	Key Output	Status
v0.1	Baseline integration	Correct best.pt integration + baseline metrics	Complete
v0.2	FP/FN diagnosis	Instance-level error counts on 3,833 validation images	Complete
v0.3	Field testing + R&D	UI findings, hard-case evidence, next experiment plan	Current

Current priorities

- Verify whether the apparent single-detection behavior is caused by the model/inference pipeline or by the UI output logic.
- Improve D40 and hard-case recall with one controlled training experiment while keeping validation and test splits unchanged.
- Fix the Detected Objects card so results remain readable in dark mode.
- Re-test the same Saudi field images before and after retraining to demonstrate measurable improvement.

2. R&D v0.1 - Baseline Integration and Validation

Problem found in the prototype

The first prototype loaded the generic pretrained yolov8n.pt model. That model detects general COCO objects such as people and vehicles and is not the project-specific road-damage detector.

Integration correction

The Streamlit application was changed to load reports/baseline/weights/best.pt, convert uploaded images to RGB, and use a confidence threshold of 0.26. This aligned the prototype with the trained four-class road-damage model: D00, D10, D20, and D40.

Reproduced full-validation metrics

Class	Precision	Recall	mAP50	mAP50-95	Instances
All	0.669	0.547	0.601	0.318	5,445
D00	0.644	0.545	0.586	0.339	2,560
D10	0.635	0.562	0.596	0.302	1,182
D20	0.706	0.643	0.703	0.389	1,061
D40	0.691	0.439	0.520	0.240	642

Baseline interpretation

D20 is the strongest class. D40 is the weakest, particularly in Recall (0.439) and mAP50-95 (0.240). This motivated the deeper FP/FN analysis in v0.2.

3. R&D v0.2 - FP/FN Error Analysis

The baseline predictions were compared against the validation ground-truth labels using confidence threshold 0.26 and IoU $\geq$ 0.50. The purpose was diagnostic: determine whether the model mainly misses damage, creates false alarms, or confuses classes.

Class	TP	FP	FN	Misclassification
D00	1,299	682	1,235	26
D10	621	315	546	15
D20	660	226	380	21
D40	261	98	375	6

What the counts mean

- True Positive: correct class and sufficient overlap with the ground-truth box.
- False Negative: a real labeled damage instance exists but no matching prediction is produced.
- False Positive: the model produces a prediction with no matching ground-truth instance.
- Misclassification: the location matches but the predicted class is different.

Key diagnosis

D40 has 375 false negatives versus 261 true positives and only 6 misclassifications. This indicates that the central D40 problem is missed detection rather than class confusion. The diagnostic D40 miss proportion is about 58.4%.

Controlled improvement direction

The next model experiment should focus on D40 representation and hard examples while leaving the validation/test data unchanged. Candidate changes include moderate D40 oversampling, realistic brightness/contrast variation, small scale/translation changes, mild blur where representative, and label-quality review. Only a small number of related factors should change in one experiment so the impact can be measured.

4. R&D v0.3 - Field Testing and Interface Findings

Manual testing of the current Streamlit prototype produced three important qualitative findings. These observations support the next R&D cycle, but they should be treated as field-testing evidence rather than formal dataset metrics.

Finding A - Dark-mode Detected Objects visibility

In light mode, the Detected Objects card is readable. In dark mode, the card can appear blank/unreadable even when a D40 bounding box is clearly shown on the image. This points to a UI/CSS contrast problem, not necessarily a model problem.

Recommended fix: use theme-aware or explicit high-contrast foreground/background colors for the result card and verify both Streamlit themes.

Finding B - Suspected single-detection behavior

During manual testing, some images appear to contain more than one damaged region while the prototype shows only one detection. This needs controlled verification before changing the model.

- If `len(results[0].boxes)` returns multiple boxes but the UI shows one, the issue is in the application rendering/listing logic.
- If the raw model also returns one box, investigate confidence threshold, Non-Maximum Suppression, training coverage, and object size/visibility.
- Use a dedicated multi-damage test set so this behavior can be reproduced consistently.

Finding C - Complex scenes are harder

Images with many visual details, vehicles, reflections, low light, distant/small damage, or complex road texture are less reliable. One tested night scene produced no detection. This is consistent with the previously measured false-negative/recall weakness.

Recommended R&D action: create a hard-case evaluation bucket tagged by condition (night, cluttered, small damage, distant damage, wet/reflection, low contrast, multiple damages) and reuse the same images after retraining.

5. Field-Test Evidence

The following screenshots are retained as qualitative evidence. They show both successful detections and failure/interface cases that should be re-tested after the next model/UI revision.



Figure A. Light mode - D40 detected at about 0.71 confidence; result card is readable.

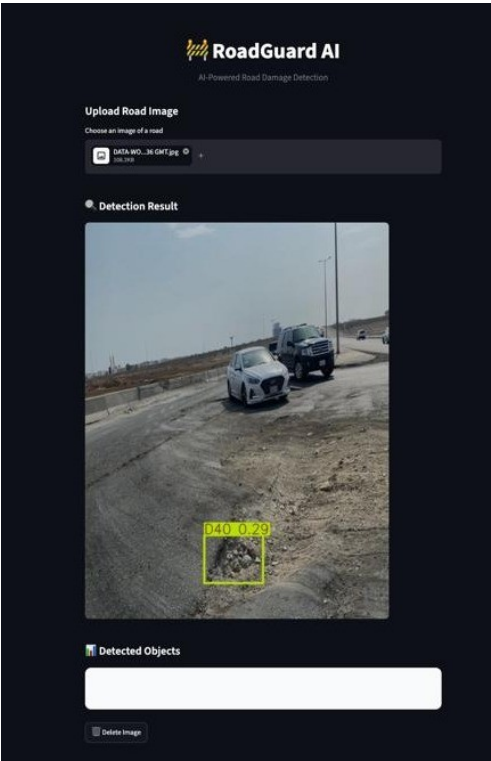


Figure B. Dark mode - D40 detected at about 0.29 confidence in a visually busy scene.

5. Field-Test Evidence (continued)

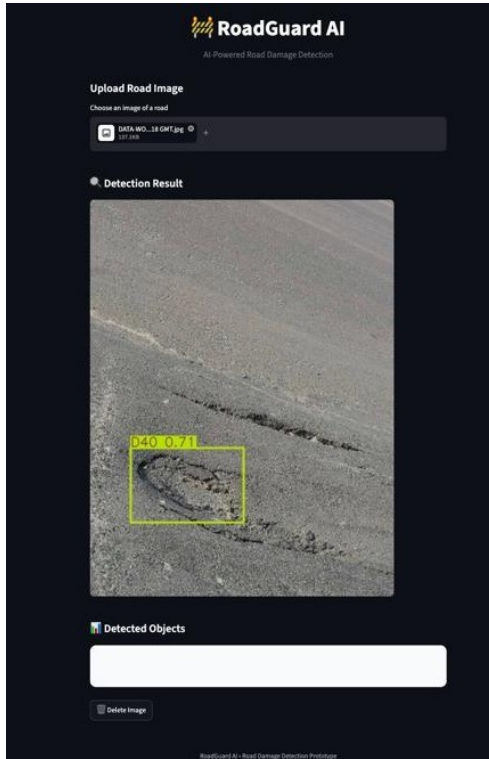


Figure C. Dark mode - D40 detected at about 0.71, while the Detected Objects card appears blank/unreadable.

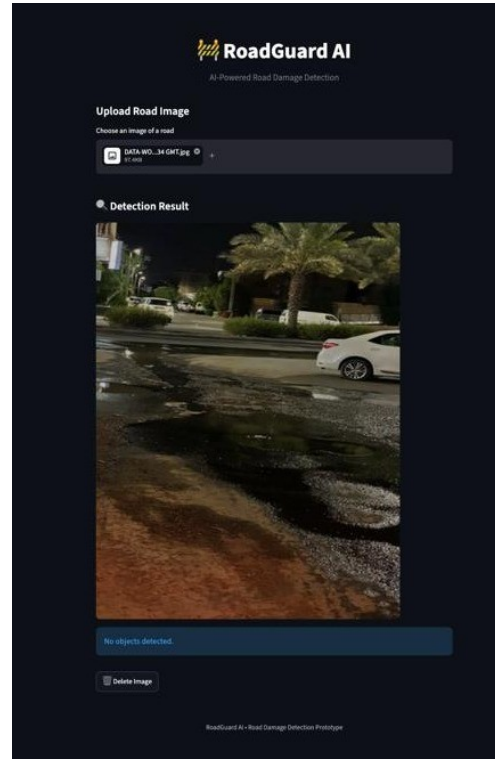


Figure D. Complex night scene - no objects detected; useful as a hard-case regression example.

How to use these screenshots

Keep the same images as a small regression set. After retraining or changing the UI, run the exact same images again and compare detected classes, box count, confidence, and result-card visibility.

6. Recommended Next R&D Cycle

Step 1 - Separate UI defects from model defects

- Fix dark-mode result-card visibility in app.py/CSS without changing model weights.
- Instrument the prototype to display the number of raw YOLO boxes returned.
- For multi-damage images, compare raw box count against boxes/rows actually rendered by the UI.

Step 2 - Build a hard-case test set

Create a small, version-controlled manifest of Saudi test images (do not mix them into train/validation) and tag each image by challenge type. The images themselves can remain outside GitHub if licensing/size is a concern; the manifest and selected permitted screenshots can be committed.

Step 3 - Run one controlled D40-focused training experiment

- Keep the same validation and test splits.
- Improve D40 representation with moderate sampling/augmentation based on observed hard cases.
- Do not overwrite the baseline best.pt; save the new weights under a new experiment name.
- Compare D40 Recall, D40 mAP50, overall mAP50, precision, and hard-case behavior against the baseline.

Success criteria

Measure	Baseline	Next-version target
D40 Recall	0.439	Improve above baseline without major precision loss
D40 mAP50	0.520	Improve above baseline
Multi-damage UI	Not verified	All raw predictions are displayed/listed
Dark-mode results	Unreadable in observed cases	Readable in light and dark mode

Recommended repository structure

Use a Git-friendly folder name such as research-and-development. The package accompanying this PDF is already organized as:

```
research-and-development/
  README.md
  CHANGELOG.md
  v0.1/baseline_integration_and_validation.md
  v0.2/fp_fn_error_analysis.md
  v0.3/field_testing_and_interface_findings.md
  v0.3/assets/...
  v0.3/reports/RoadGuard_AI_RnD_Report_v0.3.pdf
```

Versioning rule

Keep each version as a historical snapshot. When new findings are added, create v0.4 instead of rewriting v0.1-v0.3. This makes the R&D trail easy for the team and instructor to audit.